Instructions

1. There is no late submission policy for any homework.
2. You may receive assistance from any human, robot, or LLM; however, during the demo they are restricted to thoughts and prayers.
3. If you are unable to explain any part of your solution you will be awarded with bonus marks **-5**.

Exercise 1

Case Study: University Ping-Pong Championship Manager

The university sports department is organizing a live `Ping-Pong Championship`. You have been hired to design the backend management system for handling players entering the tournament arena.

There are two categories of players:

- **VIP Players** Professional or invited players who receive priority.
- **Normal Players** Regular university participants.

Example Players (Not Preloaded)

The following names are examples only. **No player exists initially.** All players appear only through `REGISTER` commands.

VIP Examples:

- Arin Bashel (Highest Level)
- Mehran Kadeer (Higher Level)
- Faris Halan Sayed (High Level)

Normal Examples:

- Ameen Rayan
- Wasir Arlen
- Alaric Rayan

Tournament Rules

1. VIP players always play before Normal players.
2. Among VIP players, the one with the higher `level` plays first.
3. If two VIP players have the same level, the one who registered earlier plays first.
4. Normal players play strictly in First-In-First-Out (FIFO) order.

To break ties among VIP players with equal level, you must maintain an internal counter:

$$arrivalOrder = 0, 1, 2, 3, \ldots$$

Each successful `REGISTER` increments this counter.

The system also provides an `UNDO` feature.

Important Rule: Only operations that change the system state are pushed to the UNDO stack. Ignored or no-op commands are **not** recorded.

Input

The first line contains an integer Q ($1 \leq Q \leq 10^5$) the number of operations.

Each of the next Q lines contains one of the following commands:

- `REGISTER id name level type`
- `PLAY`
- `BOOST id x`
- `WITHDRAW id`
- `UNDO`
- `STATUS`

Command Specifications

REGISTER id name level type

Registers a new player.

- `id` is an integer.
- `name` is a string without spaces.
- `level` is an integer.
- `type` is either `VIP` or `NORMAL`.

All IDs are guaranteed to be unique in valid test data. If a repeated ID appears, ignore the command (do not modify state).

Each successful registration assigns a unique `arrivalOrder`.

PLAY

The next eligible player plays the match and is removed.

- On success, print:

```
id name
```

- If no players are waiting, print:

```
NO PLAYERS
```

BOOST id x

Increase the level of a VIP player by x.

- If the player is NORMAL, ignore the command.
- If the player does not exist, ignore the command.

WITHDRAW id

Remove the player from the waiting list if currently present.

- If the player does not exist, ignore the command.

Auxiliary data structures (e.g., hash maps for lazy deletion or indexing) are allowed to support this efficiently.

UNDO

Undo the most recent state-changing operation:

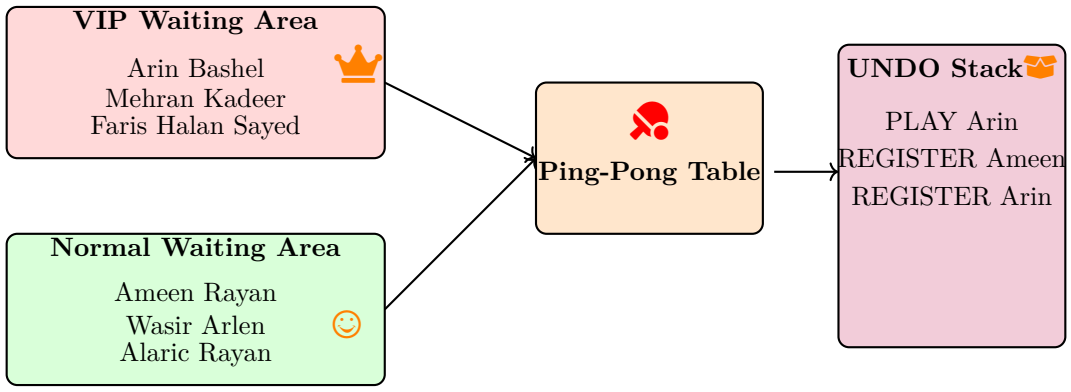
- REGISTER
- PLAY
- BOOST
- WITHDRAW

Ignored or no-op commands are not recorded.

STATUS

Print:

```
VIP_count Normal_count
```



Constraints

- $1 \leq Q \leq 10^5$
- All valid IDs are unique.
- UNDO operations are always valid.

Implementation Requirements (OOP)

You must implement the solution using proper C++ Object-Oriented Programming.

Class Design

- Create a `Player` class with private members:

$$id, name, level, type, arrivalOrder$$

- Provide constructor overloading.
- Provide getter methods.
- Provide setter for level (used in BOOST).

Required Data Structures

- Queue for NORMAL players.
- Priority Queue for VIP players (ordered by level, then arrivalOrder).
- Stack for UNDO operations.
- Auxiliary structure (e.g., `unordered_map`) for ID lookup.

Hint

- FIFO Queue
- Highest priority first Priority Queue
- Reverse most recent operation Stack
- Efficient ID access Hash map

⚠**Standard Template Library (STL) queue and stack in not allowed for exercise 01.**

Exercise 2

Sensor Data Processing System using STL Containers

You are required to design and implement a **Sensor Data Processing System** for a smart campus environment. The system monitors multiple buildings using different types of sensors (e.g., temperature, humidity).

The system must efficiently support multiple data access patterns:

- Fast lookup of a sensor by its unique ID.
- Retrieval of all sensors installed in a specific building.
- Processing emergency alerts based on priority (highest priority first).

Your task is to complete the implementation of the `SensorProcessor` class using appropriate STL containers to ensure efficient performance.

System Requirements

1. SensorReading Structure

Each sensor reading contains:

- `sensorId` (integer) Unique ID of the sensor
- `location` (string) Building name
- `sensorType` (string) Type of sensor (e.g., temperature, humidity)
- `value` (double) Current reading value

2. Alert Structure

Each alert contains:

- `priority` (integer) Higher number means higher priority
- `message` (string) Alert description
- `alertType` (string) Type of alert (INFO, WARNING, ERROR, CRITICAL)

Functional Requirements

You must implement the following member functions inside the `SensorProcessor` class:

1. **addSensorReading(const SensorReading& reading)** Store sensor data such that:
   - Lookup by sensor ID is efficient (average $O(1)$ time).
   - Sensors can be grouped and retrieved by location efficiently.
2. **addAlert(const Alert& alert)** Store alerts in such a way that:
   - Alerts with higher priority are processed first.
3. **processNextAlert()**
   - Process and display the highest-priority alert.
   - Remove it from the system.
   - If no alerts exist, display an appropriate message.
4. **findSensorById(int sensorId)**
   - Return a pointer to the sensor data.
   - If not found, return `nullptr`.
5. **getSensorsByLocation(const string& location)**
   - Return all sensors installed in the given location.
   - If none exist, return an empty vector.

Performance Expectations

Your container selection should ensure:

- Sensor ID lookup: Average $O(1)$ time
- Location-based queries: Efficient grouping and retrieval
- Alert processing: Automatic priority ordering

Edge Cases to Handle

- Searching for a non-existent sensor ID
- Querying a location with no sensors
- Processing alerts when the alert queue is empty